

TBE Custom Operator Development Guide (CLI)

Issue 01
Date 2020-05-30



HUAWEI TECHNOLOGIES CO., LTD.

Copyright © Huawei Technologies Co., Ltd. 2020. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions



HUAWEI and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Contents

1 Introduction.....	1
2 Intended Audience.....	3
3 Precautions for Operator Development.....	4
4 Environment Preparation.....	5
5 Development Workflow.....	6
6 Creating a Custom Operator Development Project.....	9
7 Developing a Custom Operator.....	11
7.1 Operator Basics.....	11
7.2 Implementing an Operator.....	16
7.2.1 Procedure.....	16
7.2.2 Importing Python Modules.....	17
7.2.3 Implementing an Operator.....	17
7.2.4 Scheduling and Building an Operator.....	19
7.2.5 Running an Operator.....	20
7.3 Running an Operator for Verification.....	20
7.3.1 Building an Operator.....	20
7.3.2 Building Test Data Files.....	21
7.3.3 Running a Single Operator.....	23
8 Developing the Plug-In of a Custom Operator.....	27
8.1 Implementing a Plug-In.....	27
8.1.1 Procedure.....	27
8.1.2 Including Header Files.....	28
8.1.3 Parsing an Operator.....	30
8.1.4 Inferring the Output Tensor Description of an Operator.....	34
8.1.5 Building an Operator.....	37
8.1.6 Registering an Operator.....	38
8.2 Sample Code.....	40
8.3 Building a Plug-In.....	40
9 Loading a Plug-In for Model Conversion.....	42
10 Developing an Application.....	45

11 Common Operations.....	46
11.1 Defining an Operator in File caffe.proto.....	46
11.2 Installing the CMake Software.....	47
12 Appendix.....	48
12.1 Change History.....	48

1 Introduction

Custom operator development involves generating and running the offline model file. In most cases, since the Ascend AI software stack supports most operators, you only need to provide the deep learning model file. The offline model file can be obtained by using the offline model generator (OMG) tool. Then, you can use Matrix to create applications. You need to develop custom operators in the following situations:

- An operator in the model is not supported by the Ascend AI software stack.
- You would like to modify the computation logic of an operator.
- You would like to develop your own operators to improve computing performance.

The Ascend AI software stack provides the tensor boost engine (TBE) framework, which enables custom operator development in Python. Operators can be developed by using TBE in the following ways:

- **TVM primitive development**
TBE introduces backend code generation to the Tensor Virtual Machine (TVM). Therefore, the development pattern of TBE is basically the same as that of the TVM. However, the computing scheduling is specific to the Da Vinci architecture. To achieve better performance, you need to master the scheduling primitive for data slicing. Therefore, this approach is recommended for developers who have deep understanding of TVM programming and the Da Vinci architecture.
- **Domain-specific language (DSL) development**
To facilitate custom operator development, the Ascend AI software stack adopts the TVM Operator Inventory (TOPI) mechanism of the TVM. The code for common computation scheduling is provided and encapsulated into APIs. Using TBE-based DSL development, you only need to declare the computing process in these DSLs, use the Auto Schedule mechanism to specify the target generation code, and then build the code into dedicated kernels.

Simply put, multiple tensors are obtained from the input of multiple tensors that go through a compute node. TVM primitive development is essentially the same as SDL development, except that the abstraction layers are different. Operators developed by using either approach are called TBE operators.

After a custom operator is developed, you need to develop a custom plug-in that provides functions for parameter parsing, shape inference, and kernel compilation.

During model conversion, OMG calls the plug-in to parse parameters and infer the tensor shape, builds the custom operator into a kernel, and inserts the kernel into the offline model.

 NOTE

This document describes how to develop TBE operators based on the SDL.

2 Intended Audience

This document is intended for developers who develop operators using TBE. After reading this document, you will be able to:

- Develop TBE operators and operator plug-ins.
- Develop custom operators based on the samples provided in this document.

To better understand this document, you should have:

- Capability to develop Python/C++/C applications
- Good understanding of mathematical expressions
- Good understanding of machine learning and deep learning
- Good understanding of the Da Vinci architecture
- Good understanding of TVM and open source framework Caffe

3 Precautions for Operator Development

Precautions for Developing Custom Operators for an SSD Network with the Post-Processing Node

The PriorBox operator and detection output operator of the SSD network are automatically integrated during model conversion. If the implementation of the PriorBox operator of the SSD network has been customized, model conversion will fail due to the integration failure. In this situation, you need to delete the detection output operator from the network model file, and customize a post-processing node in engine orchestration to implement the function of the detection output operator.

Others

- Do not modify the implementation of the built-in operators of the framework (all files are saved in **ddk/include/inc/custom**). Otherwise, exceptions may occur. For example, the system fails to be started or the model cannot be converted.
- When developing an operator plug-in, customers shall be responsible for the source code to avoid backdoor implantation.

4 Environment Preparation

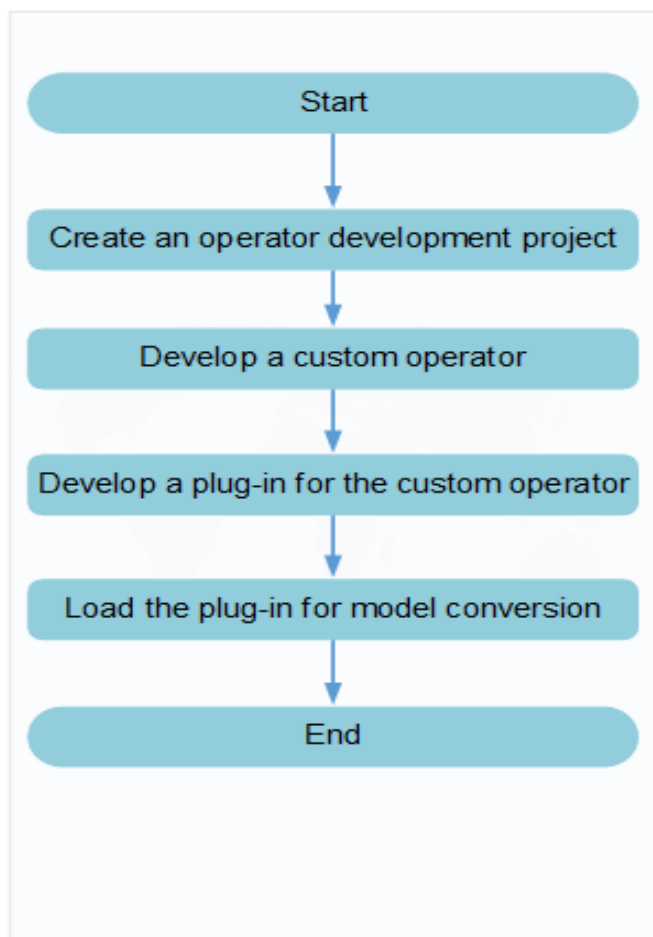
Before developing a custom operator, ensure that the software environment is well prepared:

- The development environment has been set up. That is, the DDK has been deployed, the building environment has been configured, and environment variables have been configured, according to *Development Environment Setup Guide (Linux)*.
- The sample has been deployed, which can be modified for the operator development in command line interface (CLI) mode. For details about the sample deployment, see the *DDK Sample User Guide (CLI)*.

5 Development Workflow

Figure 5-1 shows the workflow of developing a custom TBE operator.

Figure 5-1 Developing a custom TBE operator



The workflow is described as follows:

1. Create an operator development project.
You can quickly create a project in the Mind Studio IDE or adapting the DDK demo project in CLI mode.

Table 5-1 describes the differences between the two modes. This document is oriented to the CLI mode.

Table 5-1 Custom operator development modes provided by the TBE

Development Mode	Tool Dependency	Description	Build Mode	Reference Document
CLI	In this mode, the development of TBE custom operators does not depend on Mind Studio but depends only on the installed device development kit (DDK). NOTE The DDK is an Ascend AI processor-based digital development kit that contains project code samples and is integrated with library and header file dependencies, allowing project building by using Makefile.	There is no UI or automatically generated framework code. You have to develop the code for operator and operator plugin implementation yourself or modify the code sample provided by the DDK.	Create a build rule file based on the DDK code sample.	<i>TBE Custom Operator Development Guide (CLI)</i>

Development Mode	Tool Dependency	Description	Build Mode	Reference Document
Mind Studio	In this mode, operators are developed in Mind Studio. For details, see Tutorials> Custom Operator Development in <i>Mind Studio User Manual</i> .	The framework code is automatically generated based on the model file imported during project creation and selected operators. You only need to pay attention to the operator implementation code.	Create a build rule file from the configurations in Mind Studio, involving zero programming.	<i>TBE Custom Operator Development Guide (Mind Studio)</i>

2. Develop a custom operator.
Develop the operator code and run a single operator for verification.
3. Develop the custom operator plug-in. The custom operator plug-in registers the operator with Framework and provides functions for parameter parsing, shape inference, and kernel compilation.
4. Load the operator plug-in to convert the model. During model conversion, OMG calls the plug-in to parse parameters, infer the tensor shape, build the custom operator into a kernel, and insert the kernel into the offline model.

6

Creating a Custom Operator Development Project

In CLI mode, you can create your own project by simply adapting the sample project.

The sample project provides the TBE Reduction operator and its plug-in code sample. You can create your own operator project based on this sample project as follows:

Step 1 Log in to the DDK server as the common user who installs the DDK.

Step 2 Create a workspace directory.

```
mkdir $HOME/tools/projects
```

Step 3 Create a custom project directory.

```
mkdir $HOME/tools/projects/customop_te
```

Create an operator code directory.

```
mkdir $HOME/tools/projects/customop_te/operator
```

Create an operator plug-in code directory.

```
mkdir $HOME/tools/projects/customop_te/plugin
```

Step 4 Copy the sample code in the sample project to the custom project directory.

NOTE

- This example describes how to develop a custom operator by using the sample code of the built-in reduction operator and its plug-in.
- In the example, the sample installation path is **\$HOME/tools/sample**. Replace it with the actual installation path in the preceding commands.
- Copy the sample code file **reduction.py** of the reduction operator and the sample data generation script **data_gen.py** used for operator verification to the custom operator code directory.

```
cp -rf $HOME/tools/sample/customop/python/reduction.py $HOME/tools/projects/customop_te/operator/
```

```
cp -rf $HOME/tools/sample/customop/python/data_gen.py $HOME/tools/projects/customop_te/operator/
```

- Copy the sample code file of the reduction operator plug-in and the **Makefile** file to the custom operator plug-in code directory.
cp -rf \$HOME/tools/sample/customop/customop_caffe_demo/caffe_reduction_layer.cpp \$HOME/tools/projects/customop_te/plugin/
cp -rf \$HOME/tools/sample/customop/customop_caffe_demo/Makefile \$HOME/tools/projects/customop_te/plugin/
- Copy the sample Caffe network model file to the custom project directory.
cp -rf \$HOME/tools/sample/customop/customop_caffe_demo/model/\$HOME/tools/projects/customop_te/

Table 6-1 shows the directory structure of the custom operator development project.

Table 6-1 Directory structure

Sample Directory	File	Description
operator	reduction.py	Code file of the reduction operator
	data_gen.py	File generated based on the sample used for verifying the reduction operator
plugin	caffe_reduction_layer.cpp	Code file of the reduction operator plug-in
	Makefile	File defining the rules for compiling the reduction operator plug-in
model	deploy_mylenet-1.prototxt	Source mylenet model file
	mylenet-1.caffemodel	Pre-trained mylenet model file

----End

7 Developing a Custom Operator

[7.1 Operator Basics](#)

[7.2 Implementing an Operator](#)

[7.3 Running an Operator for Verification](#)

This section describes how to run a single custom operator to verify its correctness.

7.1 Operator Basics

A deep learning algorithm consists of multiple compute units, that is, operators (Ops). In Caffe, an operator describes the computation logic of the layer, for example, the convolution that performs convolution and the Fully-Connected (FC) layer that multiplies the input by a weight matrix.

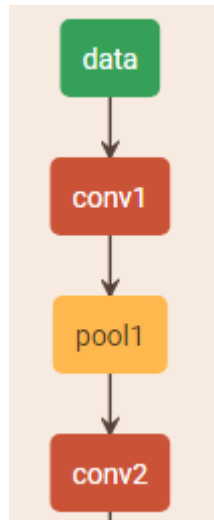
The following introduces some basic terms about operators.

Operator Type

Every operator is of a specific type, for example, convolution. A network can have different operators of the same type.

Operator Name

The name of an operator identifies the operator on a network, and therefore must be unique on a network. The example network has operators conv1, pool1, and conv2. conv1 and conv2 are of the same type convolution. conv1 and conv2 each indicates a convolution operation.

Figure 7-1 Network topology

Tensor

Tensors are used to represent the input data and output data in TBE computations. **TensorDesc** (the Tensor descriptor) describes the input data and output data. [Table 7-1](#) lists the attributes of the **TensorDesc** struct.

Table 7-1 TensorDesc attributes

Attribute	Description
name	Indexes a Tensor. The name of each Tensor must be unique.
shape	Specifies the shape of a Tensor, for example, (10) , (1024, 1024) , or (2, 3, 4) . For details, see Shape . Default value: N/A Format: (i1, i2, ... in) , where, i1 to in are positive integers.
dtype	Specifies the data type of a Tensor object. Default value: none Value range: float16, int8, int16, int32, uint8, uint16, bool NOTE - The supported data types vary with the operation. For details, see <i>TBE API Reference</i>. - TBE APIs support both the float16 and float32 types. However, OMG converts the float32 type to the float16 type during model conversion. Therefore, the current version does not support the float32 type for custom operator development.
format	Specifies the data layout format. For details, see Format .

- Shape

The shape of a Tensor is described in the format of **(D0, D1, ..., Dn - 1)**, where, **D0** to **Dn** are positive integers.

For example, shape (3, 4) indicates a 3 x 4 matrix, where the first dimension has three elements and the second dimension has four elements.

The number count in the bracket equals to the dimension count of the Tensor. The first element depends on the element count in the outer square brackets, and the second element depends on the element count in the second left square bracket, and so on. See the following examples.

Table 7-2 Tensor shape examples

Tensor	Shape
1	(0,)
[1,2,3]	(3,)
[[1,2],[3,4]]	(2,2)
[[[1,2],[3,4]], [[5,6],[7,8]]]	(2,2,2)

- Format

In the deep learning framework, n-dimensional data is stored by using an n-dimensional array. For example, a feature graph of a convolutional neural network is stored by using a four-dimensional array, including the batch size (N), feature map height (H), feature map width (W), and number of feature map channels (C), respectively.

Data is stored in linear mode only because the dimensions are arranged with a fixed layout. Different deep learning frameworks store feature graph data with different layouts. For example, Caffe uses the layout [Batch, Channels, Height, Width], that is, NCHW, while TensorFlow uses the layout [Batch, Height, Width, Channels], that is, NHWC.

For an RGB image as shown in [Figure 7-2](#), the pixel values of each channel are clustered in sequence as RRRGGGBBB with the NCHW layout. However, with the NHWC layout, the pixel values are interleaved as RGBRGBRGB.

Figure 7-2 NCHW and NHWC



To improve data access efficiency, the tensor data is in the Ascend AI software stack is stored in the 5D format NC1HWC0. C0, closely related to the micro architecture, is the size of the cube unit in the AI Core. C0 is **16** for FP16 or **32** for INT8. C0 needs to be stored contiguously. $C1 = (C + C0 - 1)/C0$. If the division result is not an integer, the last data record is padded with zeros for alignment with C0.

Steps of NHWC-to-NC1HWC0 conversion:

- a. Split the NHWC data into C1 pieces of NHWC0 along the C dimension.
- b. Arrange the C1 pieces of NHWC0 in the memory contiguously, obtaining NC1HWC0.

Operator Attributes

Different operators have different attribute values. The following describes some common operator attributes.

- Axis

An axis is denoted by the subscript of a dimension of a tensor. For a two-dimensional tensor with five rows and six columns, that is, with shape (5, 6), axis 0 represents the first dimension in the tensor, that is, the row; axis 1 represents the second dimension of tensor, that is, the column.

For example, for tensor $[[[1,2],[3,4]], [[5,6],[7,8]]]$ with shape (2, 2, 2), axis 0 represents data in the first dimension, that is, matrices $[[1,2],[3,4]]$ and $[[5,6],[7,8]]$, axis 1 represents data in the second dimension, that is, arrays $[1,2]$, $[3,4]$, $[5,6]$, and $[7,8]$, and axis 2 indicates the data in the third dimension, that is, numbers 1, 2, 3, 4, 5, 6, 7, and 8.

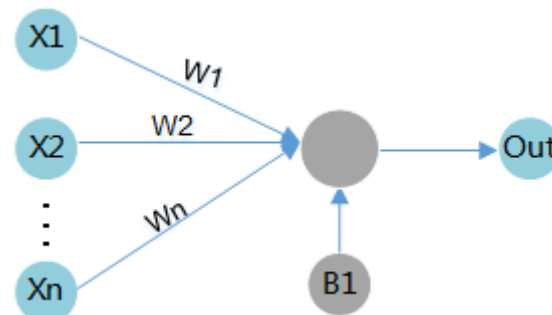
A negative **axis** is interpreted as indexing from the end.

- Bias

A bias is another linear component to be applied to the input data, in addition to a weight. The bias is added to the product of the input and its weight.

As shown in [Figure 7-3](#), in the compute unit, input X1 is multiplied by its associated weight W1 and then added with its associated bias B1, that is, $X1 * W1 + B1$.

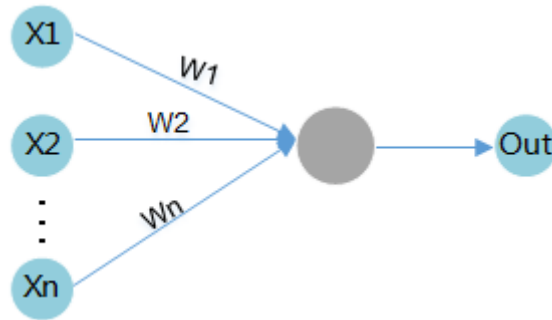
Figure 7-3 Bias computation example



- Weight

The input data is multiplied by a weight value in the compute unit. For example, for a two-input operator, an associated weight value is allocated to each of the inputs. Generally, data with more importance is assigned with a greater weight value. Therefore, the feature indicated by data with zero weight can be ignored.

As shown in [Figure 7-4](#), in the compute unit, input X1 is multiplied by its associated weight W1, that is, $X1 * W1$.

Figure 7-4 Weight computation example

Sample Operator: Reduction

Reduction is a Caffe operation that removes one or more dimensions from a tensor by performing certain operations across those dimensions.

- Attributes of the reduction operator
 - **ReductionOp**: operation type. Four operation types are supported.

Table 7-3 Operation types supported by the reduction operator

Operator Type	Description
SUM	Computes the sum of elements across specified dimensions of a tensor.
ASUM	Computes the sum of absolute values of elements across specified dimensions of a tensor.
SUMSQ	Computes the sum of squares of elements across specified dimensions of a tensor.
MEAN	Computes the mean values of elements across specified dimensions of a tensor.

- **axis**: dimensions to reduce. Reduction is performed on the dimension indicated by axis and its subsequent ones. The value range is $[-N, N - 1]$. For example, for an input tensor with shape (5, 6, 7, 8):
 - If axis = 3, the shape of the output tensor is (5, 6, 7).
 - If axis = 2, the shape of the output tensor is (5, 6).
 - If axis = 1, the shape of the output tensor is (5).
 - If axis = 0, the shape of the output tensor is (1).
- **coeff**: scalar, scaling factor. The value 1 indicates that the output is not scaled.
- Data of the reduction operator
 - Input data
 - The input contains the Tensor data and Tensor description.

Input data: Tensor **x**

The description of Tensor **x** contains the following attributes:

Table 7-4 Input of the reduction operator

Input Parameter	Description
x	Name of the input Tensor, whose shape is determined by the shape parameter
shape	Shape of the input data, N-dimensional
dtype	Type of the input data, either float16 or float32

- Output data

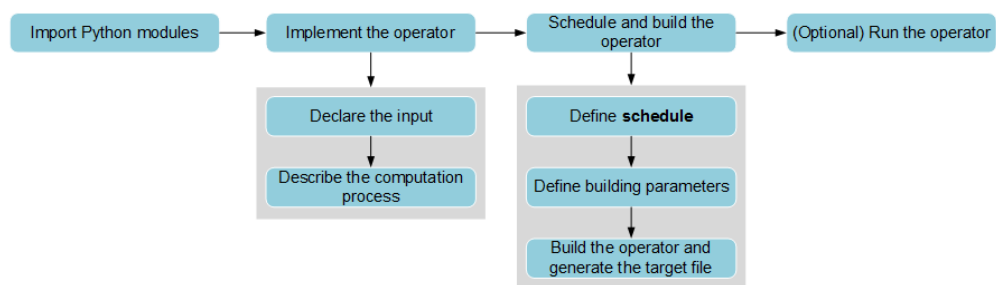
y: tensor of the identical data type as input **x**, whose shape is determined by the input Tensor shape and the specified **axis**

7.2 Implementing an Operator

7.2.1 Procedure

The code of a TBE operator is developed in Python. [Figure 7-5](#) shows the implementation procedure.

Figure 7-5 Implementing a TBE custom operator



NOTE

- The supported input data types for custom operators are as follows: float16, int8, int16, int32, uint8, uint16, and bool.
- The supported data types vary with the operation. For details, see *TBE API Reference*.
- TBE APIs support both the float16 and float32 types. However, OMG converts the float32 type to the float16 type during model conversion. Therefore, the current version does not support the float32 type for custom operator development.
- TBE provides sample code of some custom operators for user reference or direct use in **ddk/site-packages/topi-0.4.0.egg/topi/cce** in the DDK installation directory.

7.2.2 Importing Python Modules

Import the Python modules provided by the Ascend AI software stack. The sample code is provided as follows.

```
import te.lang.cce
from te import tvm
from topi import generic
```

Where,

- **te.lang.cce**: Introduces the SDL interfaces supported by TBE, including common operations such as vmuls, vadds, and matmul.
For details about the interface definition, see the Python functions in the **/site-packages/te-0.4.0.egg/te/lang/cce/** directory in the DDK installation path. For details about the usage of the Python functions, see *TBE API Reference*.
- **te.tvm**: Introduces the code generation mechanism of the TVM.
For details about the interface definition, see the Python functions in the **/site-packages/te-0.4.0.egg/te/tvm** directory in the DDK installation path. For details about how to use the Python functions, visit <https://docs.tvm.ai/>.
- **topi.generic**: Provides the operator auto_schedule interfaces.
For details about the interface definition, see the Python functions in the **/site-packages/topi-0.4.0.egg/topi/generic** directory in the DDK installation path. For details about the usage of the Python functions, see *TBE API Reference*.

7.2.3 Implementing an Operator

Function Definition for Operator Implementation

As described below, the implementation function of an operator contains the input tensor shape, data type, operator attributes, kernel name, and build and print configurations. This function is called by plug-in code and is executed when OMG converts the model.

```
def operationname(shape, dtype, attribute1, attribute2, ..., kernel_name="KernelName", need_build=True, need_print=False)
```

The parameters are described as follows:

- **shape**: input tensor shape. If an operator has multiple input tensors and each tensor has a unique shape, multiple shapes need to be defined as placeholders for the tensors. If multiple input tensors have a same shape, define one shape.
- **dtype**: data type of the input tensor.
- **attribute1, attribute2, ...**: operator attributes. Edit the code based on the operator definition.
- **kernel_name**: name of the operator in the kernel, that is, the name of the generated binary file. The value is user-defined and unique. The value can contain only uppercase letters, lowercase letters, digits, and underscores (_). Enter a maximum of 200 characters starting with a letter or underscore (_).
- **need_build**: build enable, either **True** or **False**

- **need_print**: Intermediate Representation (IR) print enable, either **True** or **False**

Instances of operator implementation function definitions are provided as follows.

Reduction operator:

```
def reduction(shape, dtype, axis, operation, coeff, kernel_name="Reduction", need_build=True,
              need_print=False)
```

Matmul operator:

```
def matmul(shape_a, shape_b, dtype, kernel_name="matmul", trans_a=False,
            trans_b=False, need_build=False, need_print=False):
```

Operator Implementation Logic

The TBE operator implementation logic is summarized as follows:

Define placeholders for input tensors, and then call the SDL interfaces in **te.lang.cce** to describe the computation process. The following is a code example:

```
data = tvm.placeholder(shape, name="data_input", dtype=inp_dtype)
with tvm.target.cce():
    cof = coeff
    data_tmp_input = te.lang.cce.vmults(data, cof) // Process the scaling parameter and multiply the input
    tmp = data_tmp_input
    res_tmp = te.lang.cce.sum(tmp, axis=axis) // Perform the summation operation on axis axis.
    res = te.lang.cce.cast_to(res_tmp, inp_dtype, f1628IntegerFlag=True) // Convert the data type.
```

Where,

- **data** indicates the input tensor, which is defined by using the **placeholder** interface of the TVM. A Tensor object is returned, indicating a group of input data.

If the operator has multiple input tensors, multiple Tensor objects need to be defined. For example:

```
tensor_a = tvm.placeholder(shape_a, name='tensor_a', dtype=dtype)
tensor_b = tvm.placeholder(shape_b, name='tensor_b', dtype=dtype)
```

- **vmults** (vector multiplication) and **sum** (summation) constitute the intermediate computation logic.
- **cast_to** is used to convert the data type. The output tensor must be of the identical data type as the input tensor. If the data type is changed during computation, you need to use the **cast_to** interface to convert the data type of the output tensor to that of the input tensor.

For example: If the data type of the input tensor is int8, it is converted to float16 for the vmults operation. In this case, the **cast_to** interface must be called to convert the data type of the output tensor from float16 to int8 after the computation logic is complete. **vmults** converts int8 values to float16, padding the decimal part with zeros. Therefore, **f1628IntegerFlag** is set to **True**. The sample code is as follows:

```
res = te.lang.cce.cast_to(res_tmp, inp_dtype, f1628IntegerFlag=True)
```

For details about how to use the **te.lang.cce.cast_to** interface, see Compute APIs in *TBE API Reference*.

- **res** indicates the output tensor, of the identical data type as the input tensor.

Before implementing the operator logic, you can customize the code for pre-processing the input data. The sample code is as follows:

```
# basic check
check_list = ["float16", "float32"]
if not (dtype.lower() in check_list):
    raise RuntimeError("Reduction only support %s while dtype is %s" % (
        ",".join(check_list), dtype))
reduction_op = ("SUM", "ASUM", "SUMSQ", "MEAN")

# axis parameter check
if type(axis) != int:
    raise RuntimeError("type of axis value should be int")
if axis >= len(shape) or axis < -len(shape):
    raise RuntimeError(
        "input axis is out of range, axis value can be from %d to %d" % (
            -len(shape), len(shape) - 1))

# operation parameter check
if operation not in reduction_op:
    raise RuntimeError("operation can only be one of SUM, ASUM, SUMSQ, MEAN")

# coeff parameter check
if type(coeff) != int and type(coeff) != float:
    raise RuntimeError("coeff must be a value")

# Preprocess
if axis < 0:
    axis = len(shape) + axis
shape = list(shape)
shape1 = shape[:axis] + [reduce(lambda x, y: x * y, shape[axis:])]
inp_dtype = dtype.lower()
```

7.2.4 Scheduling and Building an Operator

As shown in the following code, after the computation logic is defined, the Auto schedule mechanism performs auto scheduling. You can check the computation IR through the print mechanism provided by the TVM. The configuration information includes the print switch status, build switch status, operator name in the kernel, and input and output tensors.

```
sch = generic.auto_schedule(res)
config = {
    "print_ir": need_print,
    "need_build": need_build,
    "name": kernel_name,
    "tensor_list": [data, res]
}
te.lang.cce.cce_build_code(sch, config)
```

- Use the **auto_schedule** interface of **generic** to perform auto scheduling (defining **schedule**). The argument of the **auto_schedule** interface is the output tensor of the operator.
The **schedule** object defines how to efficiently execute the described computation process in hardware. That is, the related computations are mapped to the corresponding instructions on a hardware device. A **schedule** object contains an IR, which uses code similar to pseudo code to describe a computation process. You can print the object by using the parameter **need_print**.
- The input and output tensors are stored in **tensor_list**. The input and output tensors must be arranged in the input and output sequences of the operator.
For example: "**tensor_list**": [**tensor_a**, **tensor_b**, **res**], where, **tensor_a** and **tensor_b** are the input tensors, and **res** is the output tensor.
- The **cce_build_code** interface provided by **te.lang.cce** is used to build the operator based on scheduling and configuration. During operator building

when OMG converts the model, a dedicated kernel is built based on the input data shape, type, and operator parameters.

- **sch**: generated **schedule** object of the operator
- **config**: map of the compilation parameter configurations

After the compilation is complete, an operator target file *.o (the running target of the operator is the AI Core) or *.so (the running target of the operator is the AI CPU) and an operator description file *.json are generated.

7.2.5 Running an Operator

After the custom operator code is written, you can append the operator calling statement to the *.py code of the operator as follows. Construct the input data by referring to [7.3 Running an Operator for Verification](#) and use it to check the operator execution result.

The following provides an example.

```
if __name__ == "__main__":
    reduction((2, 3, 4), "float16", 1, "SUM", coeff = 2, kernel_name = "Reduction")
```

7.3 Running an Operator for Verification

This section describes how to run a single custom operator to verify its correctness.

7.3.1 Building an Operator

Build the operator code to generate the operator binary file and operator description file as follows:

Step 1 Obtain the DDK version.

Check the DDK version in the **/ddk_info** file in the DDK installation directory.

For example, run the following command in the DDK installation directory:

cat ddk_info

```
{
  "INTERFACE_VERSION": "1.1.1",
  "VERSION": "1.32.T3.B030",
  "NAME": "DDK"
}
```

In the preceding command output, the value of the **VERSION** field indicates the DDK version.

Step 2 Set the version and build the operator.

1. Run the following command in the **customop_te/operator** directory to enable the Python interaction mode:

python

2. In Python interaction mode, run the following commands in sequence to set the DDK version:

from topi.cce import te_set_version

```
import subprocess
te_set_version("1.32.T3.B030")
```

Figure 7-6 shows an example.

Figure 7-6 Setting the DDK version

```
Python 2.7.12 (default, Nov 12 2018, 14:36:49)
[GCC 5.4.0 20160609] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> from topi.cce import te_set_version
[07:23:43] src/tbe_lang/tune/plugin/plugin_mgr.cc:74: plugin type:tiling name:ml_tuning is registered successfully!
>>> import subprocess
>>> te_set_version("1.32.T3.B030")
'success'
>>>
```

3. Run the following commands to build **reduction.py** to generate the binary file and description file of the operator:

```
subprocess.call("python reduction.py",shell=True)
```

Figure 7-7 Generating operator files

```
>>> subprocess.call("python reduction.py",shell=True)
[06:55:46] src/tbe_lang/tune/plugin/plugin_mgr.cc:86: plugin type:tiling name:ml_tuning is registered successfully!
>
```

4. Exit Python interaction.

```
quit(0)
```

NOTE

The preceding commands are common commands for single-operator building. You can encapsulate the commands into a script for future use.

Step 3 After the operator is built, the **kernel_meta** folder is generated in the operator directory. This folder contains the operator binary file *.o (if the operator runs on AI Core) or *.so (if the operator runs on AI CPU) and operator description file *.json.

- The *.o or *.so file is the binary file of the operator.
- The *.json file is the operator description file, which is used to define operator properties and resources required for operator running.

The *.json file is parsed as follows:

```
{
  "binFileName": ".Reduction",           // Name of the binary file of the operator
  "binFileSuffix": ".o",                 // Extension of the generated operator binary file. If the operator runs on
  // AI CPU, the generated binary file name is suffixed by .so.
  "blockDim": 1,                         // Number of AI Cores used for computation
  "kernelName": "Reduction_kernel0",     // Name of the kernel function of the operator
  "magic": "RT_DEV_BINARY_MAGIC_ELF",    // AI Core running target. magic =
  // RT_DEV_BINARY_MAGIC_ELF_AICPU indicates the AI CPU running target.
  "sha256": "d5158df2ff2e64743eec7f527ddd9078b99c1d670f5adbd8b789224657ab0f91" // *.o file
  // encription value
}
```

----End

7.3.2 Building Test Data Files

Operator data includes input data, expected output data, and actual output data. For details, see [Table 7-5](#).

Table 7-5 Description of the input and output data of the operator

Category	Description
Input	Input data The input data type is either float16 or float32 for single-operator verification described in this topic.
Expect	Expected output data, used to measure the actual output • Precision Deviation: allowed precision deviation. The value range is [0, 1). The default value is 0.2. A smaller value indicates higher precision. • Statistical Discrepancy: statistical discrepancy, that is, the percentage of the data whose precision deviation is above the threshold. The value range is (0, 1). A smaller value indicates higher precision.
Output	Output data and output file path • Path: output file, generated to the output folder • Size: size of the output data, in bytes • Data Type: output data type, either 1 (float16) or 2 (float32)

The sample project provides a test data generation script **customop_te/operator/data_gen.py** for the reduction operator. Run the following command in the **customop_te/operator/** directory to generate the test data for the sample operator Reduction:

python data_gen.py reduction

Table 7-6 describes the generated test data files.

Table 7-6 Description of generated test data files

Data File	Description
Reduction_input_2_3_4_sum_axis_1.data	Indicates the input data file in binary format of the reduction operator.
Reduction_input_2_3_4_sum_axis_1.txt	Displays the binary file of the reduction operator in .txt format, facilitating result viewing.
Reduction_output_2_3_4_sum_axis_1.data	Indicates the output data verification file in binary format of the reduction operator, which is used to verify whether the output of the operator is correct after the operator runs.
Reduction_output_2_3_4_sum_axis_1.txt	Displays the binary file of the reduction operator in .txt format, facilitating result viewing.

7.3.3 Running a Single Operator

The sample provides the common code for running a single operator. You can build the code to generate executable binary files and use them for verification.

Step 1 Generate a binary file for single-operator verification in the sample project **customop_runner**.

1. Assign the write permission to the **customop_runner** sample project.
chmod -R +w \$HOME/tools/sample/customop/customop_runner/
\$HOME/tools/sample/ is the deployment path of the sample.

2. Go to the **build** directory of the sample project **customop_runner**.

cd \$HOME/tools/sample/customop/customop_runner/build

3. Generate the **Makefile** file.

If the device is Atlas 200 DK, run the following command:

cmake -Dtarget=OI .

If the device is Atlas 300, run the following command:

cmake .

4. Run the **make** command to generate a binary file.

make

The executable file **main** and dynamic library **libcustom_engine.so** are generated in **customop_runner/out/**.

NOTE

If the following message is displayed when you run the **cmake** command:

cmake: command not found...

CMake is not installed in the current system. Install the software by referring to [11.2 Installing the CMake Software](#).

Step 2 Copy the **out** directory to the directory of the TBE custom operator project.

Command example:

cp -rf \$HOME/tools/sample/customop/customop_runner/out \$HOME/tools/projects/customop_te/

Step 3 Configure the input data, output data, and verification description file.

Go to the **out** folder in the directory of the TBE custom operator project.

cd \$HOME/tools/projects/customop_te/out

- In the **input.txt** file, configure the path and name of the input data file.

The following provides an example.

dataPath=./operator/Reduction_input_2_3_4_sum_axis_1.data

If the custom operator has multiple inputs, you need to define multiple **.txt** files according to **input.txt**, for example, **input1.txt**, **input2.txt**, and more.

- Configure the output data in the **output.txt** file.

The following provides an example.

size=4
dataPath=./output/out0.data
dtype=1

- **size**: expected size of the output data file, in bytes
- **dataPath**: path and name of the output data file
- **dtype**: output data type. The value **1** indicates **float16**, and the value **2** indicates **float32**.
- In the **expect.txt** file, configure the path and file name of the data to be generated.

The following provides an example.

```
dataPath=./operator/Reduction_output_2_3_4_sum_axis_1.data
```

Step 4 Run a single operator and compare the results.

For device type Atlas 200 DK:

1. Copy the **operator** and **out** folders to the Host of the developer board as the **HwHiAiUser** user. Place them in the same directory.
For example, copy **operator** and **out** folders to the **/home/HwHiAiUser/projects** directory.
2. Log in to the Host of the developer board as the **HwHiAiUser** user in SSH mode and go to the **out** directory, for example, **/home/HwHiAiUser/projects/out**.
3. Create the **output** folder in the **out** directory for storing the generated data files. The file path and name are defined in the **output.txt** file.

```
mkdir output
```

4. Run the following command to assign the execution permission to the **main** file:

```
chmod +x main
```

5. Run the following command to run a single operator and compare the results:
./main -i input.txt -o output.txt -e expect.txt -b ../operator/kernel_meta/Reduction.o -p 0.2 -d 0.2 -k Reduction_kernel0 -t 0

The parameters are described as follows:

- **-i**: input data configuration file, which specifies the path and file name of the input data.
If there are multiple input data files, separate them with commas (,), for example, **-i input.txt,input1.txt**.
- **-o**: configuration file of the output data, which specifies the size, path, file name, and data type of the output data
- **-e**: configuration file of the expected data, which specifies the size, path, file name, and data type of the expected data
- **-b**: *.o (if the operator runs on AI Core) or *.so (if the operator runs on AI CPU) operator binary file
- **-p**: allowed precision deviation. The value range is [0, 1). A smaller value indicates higher precision.
- **-d**: statistical discrepancy, that is, the percentage of the data whose precision deviation is above the threshold. The value range is (0, 1). A smaller value indicates higher precision.
- **-k**: kernel function name. The kernel function name of the TBE operator must be the same as the value of **kernelName** in the .json file.

- **-t**: The value **0** indicates a TE_AI Core operator, and the value **1** indicates a TE_AI CPU operator.

You can check whether the TBE operator runs on the AI Core or AI CPU based on the value of **magic** in the *.json file generated after the operator is built.

After the command is executed successfully, the **out0.data** result file and the **verifyResult.txt** verification result file are generated in the **output** folder.

The following is an example of the **verifyResult.txt** file, indicating that the actual output data is consistent with the expected output data.

```
Output file ./output/out0.data compare result true
```

For device type Atlas 300:

1. Go to the **out** directory of the TBE project.
If the DDK installation user is not **HwHiAiUser**, switch to the **HwHiAiUser** user and grant the read, write, and execute permissions of the **out** directory to the **HwHiAiUser** user.
2. Create the **output** folder in the **out** directory for storing the generated data files. The file path and name are defined in the **output.txt** file.
`mkdir output`
3. Run the following command to run a single operator and compare the results:
`./main -i input.txt -o output.txt -e expect.txt -b ../operator/kernel_meta/Reduction.o -p 0.2 -d 0.2 -k Reduction_kernel0 -t 0`

The parameters are described as follows:

- **-i**: input data configuration file, which specifies the path and file name of the input data.
If there are multiple input data files, separate them with commas (,), for example, **-i input.txt,input1.txt**.
- **-o**: configuration file of the output data, which specifies the size, path, file name, and data type of the output data
- **-e**: configuration file of the expected data, which specifies the size, path, file name, and data type of the expected data
- **-b**: *.o (if the operator runs on AI Core) or *.so (if the operator runs on AI CPU) operator binary file
- **-p**: allowed precision deviation. The value range is [0, 1). A smaller value indicates higher precision.
- **-d**: statistical discrepancy, that is, the percentage of the data whose precision deviation is above the threshold. The value range is (0, 1). A smaller value indicates higher precision.
- **-k**: kernel function name. The kernel function name of the TBE operator must be the same as the value of **kernelName** in the .json file.
- **-t**: The value **0** indicates a TE_AI Core operator, and the value **1** indicates a TE_AI CPU operator.

You can check whether the TBE operator runs on the AI Core or AI CPU based on the value of **magic** in the *.json file generated after the operator is built.

After the command is executed successfully, the **out0.data** result file and the **verifyResult.txt** verification result file are generated in the **output** folder.

The following is an example of the **verifyResult.txt** file, indicating that the actual output data is consistent with the expected output data.

```
Output file ./output/out0.data compare result true
```

----**End**

8 Developing the Plug-In of a Custom Operator

[8.1 Implementing a Plug-In](#)

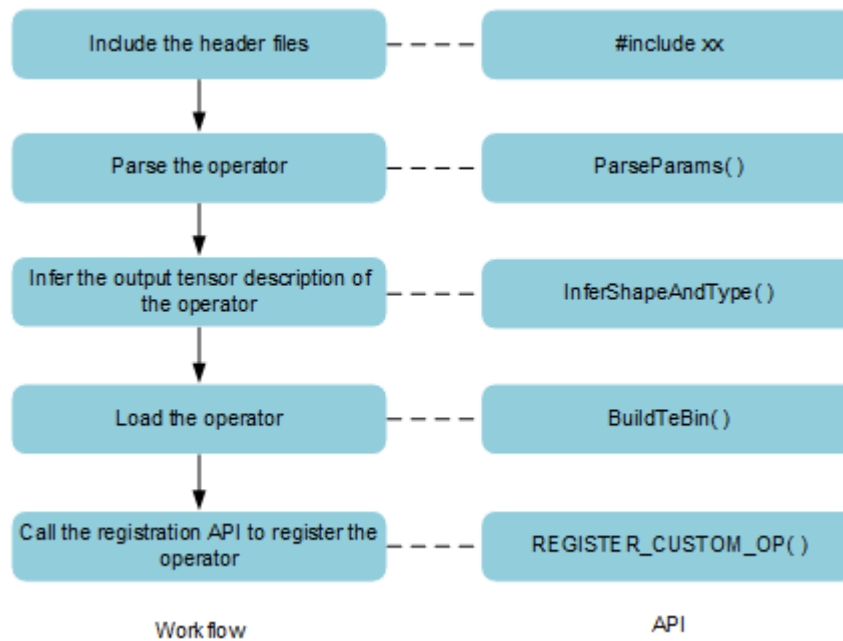
[8.2 Sample Code](#)

[8.3 Building a Plug-In](#)

8.1 Implementing a Plug-In

8.1.1 Procedure

After the custom operator is developed, OMG should be able to adapt the attribute values of the custom operator to the offline model, and the custom operator should be able to be used for running the offline model. Therefore, you need to develop the custom operator plug-in for parsing operator attributes, inferring the shapes and types, and registering the custom operator through the registration mechanism provided by OMG.

Figure 8-1 Implementing a custom operator plug-in

8.1.2 Including Header Files

Use the **#include** command in the header of the plug-in implementation file to include the header files related to the plug-in implementation functions in the plug-in implementation source file.

```
#include "custom/custom_op.h"
#include "framework/omg/register.h"
#include "framework/omg/omg_types.h"
#include "proto/caffe/caffe.pb.h"
#include "operator.h"
#include "attr_value.h"
#include <memory>
#include <string>
#include <vector>
```

Table 8-1 Description of header files

Header File	Content	Description
custom/custom_op.h	/include/inc/custom/custom_op.h in the DDK installation directory	User-defined functions for building, operator debugging, and operator verification can be called once this header file is included.
framework/omg/register.h	/include/inc/framework/omg/register.h in the DDK installation directory	Class operator registration can be used and APIs of class operator registration can be called once this header file is included.

Header File	Content	Description
framework/omg/omg_types.h	/include/inc/framework/omg/omg_types.h in the DDK installation directory	TBE custom operator information structure TEBinInfo can be used once this header file is included.
proto/caffe/caffe.pb.h	proto/caffe/caffe.pb.h generated in the directory of the operator project during the building of an operator plug-in	When the operator plug-in is built, the /include/inc/custom/proto/caffe/caffe.proto file in the DDK installation directory is automatically built, and the proto/caffe/caffe.pb.h file is generated in the directory of the operator project for the plug-in code to call to parse operator parameters.
operator.h	include/inc/graph/operator.h in the DDK installation directory.	APIs for setting/obtaining operator attributes and setting input/output can be called once this header file is included.
attr_value.h	/include/inc/graph/attr_value.h in the DDK installation directory	Data types of class AttrValue can be used once this header file is included.
memory	C++ standard library	Smart pointers, memory allocators, temporary functions for allocating and releasing dynamic memory, and functions for constructing memory objects in the C++ standard library can be called once this header file is included.
string	C++ standard library	Class string can be used to construct objects APIs of class string can be called once this header file is included.

Header File	Content	Description
vector	C++ standard library	Vector templates can be used and APIs of class vector can be called once this header file is included.

Before the operator plug-in is built, ignore the following message (the project does not have the **proto/caffe/caffe.pb.h** file).

Figure 8-2 Message indicating a caffe.pb.h parsing failure

```
#include "custom/custom_op.h"
#include "framework/omg/register.h"
#include "framework/omg/omg_types.h"
#include "proto/caffe/caffe.pb.h"
#include "operator.h"
#include "attr_value.h"
#include <memory>
#include <string>
#include <vector>
```

8.1.3 Parsing an Operator

For a newly developed custom operator, you need to customize a function for parsing operator attributes and converting the operator attribute definitions in the source model to the operator attribute definitions in the offline model supported by the Ascend AI processor. If you are rewriting a built-in operator of the Ascend AI processor, skip this section. A built-in operator is automatically parsed by its plug-in.

Function Declaration

The operator parsing function is declared as follows:

```
Status ParseParamsxx(const Message* op_origin, ge::Operator& op_dest)
```

- **ParseParamsxx**: function name, which is user-defined and must be unique
- **op_origin**: source operator model. It is a data struct in the protobuf format. It is derived from the .proto file of the Caffe model in the **/include/inc/custom/proto/caffe/caffe.proto** directory under the DDK installation directory. If the custom operator is not defined in the **caffe.proto** file, add the operator definition by referring to [11.1 Defining an Operator in File caffe.proto](#).

The operator plug-in reads operator attributes based on the operator name from the **proto/caffe/caffe.pb.h** file and **caffe.pb.cc** file generated after **caffe.proto** building and parses the operator attributes to convert the operator data structure to a data structure supported by the offline model.

Find the preset **caffe.proto** file in **/include/inc/custom/proto/caffe/caffe.proto** in the DDK installation directory. You can modify the file and add the definition of the custom operator.

- **op_dest**: target operator model. As the operator data struct of the offline model supported by the Ascend AI processor, it stores operator information. For details about class **Operator**, see **Class Operator** in *Framework API Reference*.

Procedure

Implement the **ParseParamsxx** function as follows:

- Step 1** Define the object that points to **LayerParameter** and obtain the handle to the current operator layer.

```
const caffe::LayerParameter* layer = dynamic_cast<const caffe::LayerParameter*>(op_origin);
const caffe::xxxParameter& param = layer->reduction_param();
```

Where,

- **xxxParameter** in **caffe::xxxParameter** of the **param** object must be the same as the type declared in the **LayerParameter** object.
- The name of the member function **xxx_param()** of the **layer** object must be the same as the object name declared in the **LayerParameter** object.

The following uses the Reduction and convolution operators in **caffe.proto** as an example:

```
message LayerParameter {
  optional ReductionParameter reduction_param = 136;
  optional ConvolutionParameter convolution_param = 106;
  ...
}
```

The code for obtaining the handles to the **Reduction** and **Convolution** operator layers are as follows:

```
const caffe::ReductionParameter& param = layer->reduction_param()
const caffe::ConvolutionParameter& param = layer->convolution_param()
```

- Step 2** Parse the operator attributes and assign the attributes to the **op_dest** object of the **Operator** type.

You can call the **CreateFrom<AttrValue::T>(DT&& val)** API to convert **DT** parameters to **T** parameters of class **AttrValue** and call the **SetAttr(const string& name, const AttrValue& value)** API to assign the converted values of the **AttrValue** object to the corresponding attributes of the **op_dest** object.

Type **T** is introduced to the Ascend AI software stack to simplify the type definition. The supported data types are renamed. For the mapping between type **T** and the source data type, see [Table 8-2](#). For the prototype definitions, see the **include/inc/graph/attr_value.h** file in the DDK installation directory.

Table 8-2 Mapping between type **T** and source data types

Type T	Source Data Type
INT	int64_t
FLOAT	float

Type T	Source Data Type
STR	std::string
TENSOR	TensorPtr
TENSOR_DESC	TensorDesc
GRAPH	ComputeGraphPtr
BYTES	Buffer
NAMED_ATTRS	NamedAttrs
BOOL	bool
LIST_INT	vector<INT>
LIST_FLOAT	vector<FLOAT>
LIST_BOOL	vector<BOOL>
LIST_STR	vector<STR>
LIST_TENSOR	vector<TENSOR>
LIST_TENSOR_DESC	vector<TENSOR_DESC>
LIST_GRAPH	vector<GRAPH>
LIST_BYTES	vector<BYTES>
LIST_NAMED_ATTRS	vector<NAMED_ATTRS>

For details about the **SetAttr** API, see *Framework API Reference*.

The following are examples of parsing common parameters:

- Parameters of the int or float type

For example, the operator parameters in the **caffe.proto** file are defined as follows:

```
message ReductionParameter {
...
  optional int32 axis = 2 [default = 0];
  optional float coeff = 3 [default = 1.0];
}
```

Call the **SetAttr** API to assign the value of **param.axis()** to the **axis** attribute of the **op_dest** object and convert the type to INT. Assign the value of **param.coeff()** to the **coeff** attribute of the **op_dest** object and convert the type to FLOAT, as follows:

```
op_dest.SetAttr("axis", AttrValue::CreateFrom<AttrValue::INT>(param.axis()));
op_dest.SetAttr("coeff", AttrValue::CreateFrom<AttrValue::FLOAT>(param.coeff()));
```

In the preceding information, **CreateFrom<AttrValue::T>(DT&& val)** is used to convert **DT** parameters to **T** parameters of the **AttrValue** class.

- Parameters of the enum type

For example, the operator attributes in the **caffe.proto** file are defined as follows:

```
message ReductionParameter {
  enum ReductionOp {
    SUM = 1;
    ASUM = 2;
    SUMSQ = 3;
    MEAN = 4;
  }
  ...
}
```

- a. Convert parameters of the enum type to parameters of the map type.

```
std::map<caffe::ReductionParameter_ReductionOp, std::string> operation_map = {
  { caffe::ReductionParameter_ReductionOp_SUM, "SUM" },
  { caffe::ReductionParameter_ReductionOp_ASUM, "ASUM" },
  { caffe::ReductionParameter_ReductionOp_SUMSQ, "SUMSQ" },
  { caffe::ReductionParameter_ReductionOp_MEAN, "MEAN" },
};
```

- b. Call the **SetAttr** API to assign the *operation_map* parameter of the map type to the **operation** attribute of the **op_dest** object.

```
op_dest.SetAttr("operation",
  AttrValue::CreateFrom<AttrValue::STR>(operation_map[param.operation()]));
```

For details about the **SetAttr** API, see *Framework API Reference*.

- Parameters of the repeated type

For example, the operator parameters in the **caffe.proto** file are defined as follows:

```
message xxxParameter {
  ...
  repeated float min_size = 1;
  repeated uint32 offset = 2;
  ....
}
```

- a. Convert parameters of the repeated float type to parameters of the vector<float> type, convert parameters of the repeated uint32 type to parameters of the vector<uint32> type, and assign values to the parameters of the vector type.

```
vector<float> min_size;
vector<uint32> offset;
for(int i = 0; i < param.min_size_size(); ++i)
{
  min_size.push_back(param.min_size(i)); // Call the push_back function of the
vector type to assign a value to the min_size parameter of the repeated object.
}
for(int i = 0; i < param.offset_size(); ++i)
{
  offset.push_back(param.offset(i)); // Call the push_back function in the vector
object to assign a value to the offset parameter of the repeated object.
}
```

- b. Call the **SetAttr** API to assign the value of the **min_size** parameter of the vector<float> type to the **min_size** attribute of the **op_dest** object. Assign the value of the **offset** parameter of the vector<uint32> type to the **offset** attribute of the LIST_INT type in the **op_dest** object.

```
op_dest.SetAttr("min_size",
  ge::AttrValue::CreateFrom<ge::AttrValue::LIST_FLOAT>(min_size));
op_dest.SetAttr("offset", ge::AttrValue::CreateFrom<ge::AttrValue::LIST_INT>(offset));
```

For details about the **SetAttr** API, see *Framework API Reference*.

----End

8.1.4 Inferring the Output Tensor Description of an Operator

Infer the output tensor description of the operator based on the input tensor description, operator logic, and operator attributes. The output tensor description includes the tensor shape, data type, and data layout format. In this way, all tensors can be statically allocated with memory during offline model conversion, thereby avoiding overhead caused by dynamic memory allocation.

Function Declaration

The function is declared as follows:

```
Status InferShapeAndTypexx(const ge::Operator& op, vector<ge::TensorDesc>& v_output_desc)
```

- **InferShapeAndTypexx**: function name, which is user-defined and must be unique
- **op**: compute node definition, which stores the input tensor description and operator attributes. For details about the `ge::Operator` type, see **Class Operator** in *Framework API Reference*
- **v_output_desc**: output tensor description of the compute node, including the shape, data layout format, and data type. For details about the **TensorDesc** type, see **Class TensorDesc** in *Framework API Reference*

The following describes the implementation of the **InferShapeAndTypexx** function in different scenarios.

Operator with Same-Shape Output and Input Tensors

For an operator whose output and input tensors have the identical shape, you can directly insert the description of the input tensor into the vector space in which the output tensor description is located.

A code sample is provided as follows:

```
v_output_desc.push_back(op.GetInputDesc(0));
```

The **GetInputDesc** API is used to obtain the input tensor description based on the operator input name or input index in class `Operator`. For details about the API, see **Class Operator** in *Framework API Reference*.

Operator with Reduced Dimensions

For common dimension reduction operations such as Reduction and Reduce, compute the shape of the output tensor (including the output tensor dimensions and element count of each dimension) according to information such as the operator input attribute **axis**, and then insert the shape of the output tensor into the **v_output_desc** vector.

A code sample is provided as follows:

Step 1 Obtain the input tensor description and shape of the input tensor of the operator.

```
auto tensorDesc = op.GetInputDesc(0); // Obtain the input tensor description, including the
auto shape = tensorDesc.GetShape(); // Obtain the shape of the input tensor.
```

For details about the `GetShape` interface, see **Class TensorDesc** in *Framework API Reference*.

- Step 2** Obtain the attribute values of the operator according to the computation logic, and compute the shape of the output tensor of the operator.

For example, for the Reduction operator in the mylenet network, because the upper layer of Reduction is Softmax and the output from Softmax is padded to 4-dimensional from 2-dimensional in the offline model, you need to adjust **axis** so that it points to a 2-dimensional position. After the reduce operation, and assign the adjusted **Shape** to the output tensor description.

- Obtain the key-value pair of the **axis** attribute from the operator object, obtain the **axis** attribute value from the key-value pair, convert the attribute from the INT type to the int64_t type, assign the attribute value to the variable **axis**, verify and adjust the value of **axis**, and point the value to axis 1, as follows:

```
int64_t axis = -1;
ge::AttrValue axisAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("axis", axisAttrValue)) || (ge::GRAPH_SUCCESS != axisAttrValue.GetValue<AttrValue::INT>(axis)))
{
    printf("Get axis failed!\n");
}
// In the OM model, all shape are supplemented to 4d. In this case, axis needs to be repaired to point to the original 2d.
if (axis < 0) axis -= 2;

if (axis < 0) axis += shape.GetDimNum();

if (axis < 0 || axis >= shape.GetDimNum())
{
    printf("invalid axis:%d, dim_size:%d\n", (int32_t)axis, (int32_t)shape.GetDimNum());
    return PARAM_INVALID;
}
```

- Adjust **Shape** and set the dimensions from axis 1 to 1. For example, if the input tensor is with shape (2, 3, 4, 5), adjust the shape to (2, 1, 1, 1).

```
int32_t dimsize = (int32_t)shape.GetDimNum();
int32_t idx = 0;
for(idx=axis; idx<dimsize; idx++)
{
    shape.SetDim(idx, 1);
}
```

- Set the adjusted shape to the **tensorDesc** object.

```
tensorDesc.SetShape(shape);
```

For details about APIs **GetDimNum** and **SetDim**, see **Class Shape** in *Framework API Reference*.

- Step 3** Set the output tensor description of the operator.

```
v_output_desc.push_back(tensorDesc)
```

Assign **tensorDesc** to the description object **v_output_desc** of the output tensor.

----End

Network Having Multiple Operators of the Identical Type

A network can have multiple operator layers of the same type, such as the convolution operator. Sometimes, you need to customize the shape of an operator layer (redefine an existing operator). In this case, the output tensor description inference needs to be performed according to different situations of the network, determine the operator layers to be customized based on the attributes such as

num_output, **kernel**, **stride**, and **pad**, and obtain the tensor information of the operators.

A code sample is provided as follows:

- Step 1** Assign the input tensor description to the output tensor description, and obtain the input tensor description and the shape of the input tensor of the operator.

```
v_output_desc.push_back(op.GetInputDesc(0)); // Assign the input tensor description to the
output tensor description object. Alternatively, assign the shape obtained after inference to the
tensorDesc object, and then assign the tensorDesc object to the output tensor description
object.
auto tensorDesc = op.GetInputDesc(0); // Obtain the input tensor description, including the
shape, data layout format, and data type.
auto shape = tensorDesc.GetShape(); // Obtain the shape of the input tensor.
```

For details about the GetShape interface, see **Class TensorDesc** in *Framework API Reference*.

- Step 2** Obtain the attribute values of the operator according to the computation logic, match the operator according to the attribute values of the operator and shape, and compute the shape of the output tensor of the operator.

For example, for operators of the convolution type in a network, match the convolution operator whose **num_outputs** is **128**, **shape.GetDim(0)** is **1**, **shape.GetDim(1)** is **128**, **shape.GetDim(2)** is **28**, and **shape.GetDim(3)** is **28**, and reassign the shape in the output tensor description of the operator.

- Obtains the value of **num_outputs**.

```
ge::AttrValue num_outputsAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("num_output", num_outputsAttrValue)) ||
    (ge::GRAPH_SUCCESS != num_outputsAttrValue.GetValue<AttrValue::INT>(num_outputs)))
{
    printf("GetOpAttr num_outputs failed!\n");
}
```

- Match the convolution operator whose **num_outputs** is **128**, **shape.GetDim(0)** is **1**, **shape.GetDim(1)** is **128**, **shape.GetDim(2)** is **28**, and **shape.GetDim(3)** is **28**, and reassign the shape in the output tensor description of the operator.

```
if(shape.GetDim(0) == 1 && shape.GetDim(1) == 128 &&
    shape.GetDim(2) == 28 && shape.GetDim(3) == 28 && num_outputs == 128)
{
    shape.SetDim(0, 1);
    shape.SetDim(1, 128);
    shape.SetDim(2, 28);
    shape.SetDim(3, 28);
    v_output_desc[0].SetShape(shape);
    return SUCCESS;
    return FAILED;
}
```

For details about APIs **GetDimNum** and **SetDim**, see **Class Shape** in *Framework API Reference*.

NOTE

To use **op_name** for operator matching, obtain **op_name** as follows:

```
auto op_name = op.GetName();
```

The method for obtaining other attributes **kernel_w**, **kernel_h**, **stride_w**, **stride_h**, **pad_w**, and **pad_h** are similar. You only need to change the value of **key** in **op.GetAttr**.

----End

8.1.5 Building an Operator

Function Declaration

The operator building function is declared as follows:

```
Status BuildTeBinxx(const ge::Operator& op, TEBinInfo& te_bin_info)
```

Where,

- **BuildTeBinxx**: function name, which is user-defined and must be unique
- **op**: target operator model. As the operator data struct of the offline model supported by the Ascend AI processor, it stores operator information. For details about class **Operator**, see **Class Operator** in *Framework API Reference*.
- **te_bin_info**: path of the operator binary file, operator description file path, and DDK version information. For details about the **TEBinInfo** struct, see **TEBinBuildFn** in *Framework API Reference*.

Procedure

The operator building function is called during model conversion by OMG as follows:

- Obtain the operator tensor description and operator attributes. During model conversion, the information must be fixed values for operator matching.

For example, in model conversion, match the Reduction operator whose **axis** is **1** and **Dim** of the input tensor **Shape** is **4**.

```
// Parse the operator attribute operation.
ge::AttrValue operationAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("operation", operationAttrValue)) || (ge::GRAPH_SUCCESS !=
operationAttrValue.GetValue<AttrValue::STR>(operation)))
{
    printf("GetOpAttr operation failed!\n");
}

// Parse the operator attribute axis, and adjust axis to point to the actual output dimension of the
Softmax operator at the upper layer of the Reduction operator in the mylenet network, that is, axis 1.
ge::AttrValue axisAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("axis", axisAttrValue)) || (ge::GRAPH_SUCCESS !=
axisAttrValue.GetValue<AttrValue::INT>(axis)))
{
    printf("GetOpAttr axis failed!\n");
}
// In the OM model, all shape are supplemented to 4d. In this case, axis needs to be repaired to
point to the original 2d.
if(axis < 0)
    axis -= 2;

// Parse the operator attribute coeff.
ge::AttrValue coeffAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("coeff", coeffAttrValue)) || (ge::GRAPH_SUCCESS !=
coeffAttrValue.GetValue<AttrValue::FLOAT>(coeff)))
{
    printf("GetOpAttr coeff failed!\n");
}
// Obtain the input tensor description of the operator.
TensorDesc input_desc = op.GetInputDesc(0);

// Parse the input shape and check whether Dim of the operator is 4.
if(input_desc.GetShape().GetDimNum() != 4)
```

```
{
    printf("The shape size is %d, which is not 4!", (int32_t)input_desc.GetShape().GetDimNum());
    return FAILED;
}
```

- Specify the operator implementation file, operator implementation function, and operator name in the kernel.

```
FilePath = "project_path/operator/reduction"; // Absolute path of the operator implementation file
+ name of the .py operator file
FuncName = "Reduction"; // Name of the operator implementation function in the operator
implementation file
KernelName = "Reduction"; // kernel_name defined in the operator implementation function of
the operator implementation file, that is, the name of the generated binary file
```

- Configure the path of the generated *.json operator description file.

```
te_bin_info.json_file_path = "./kernel_meta/" + KernelName + ".json";
```

During model conversion, operator information will be obtained from the operator description file in this path.

When the **omg** model conversion command is executed, the **kernel_meta** folder generated after operator building is copied to the path where the **omg** command is executed based on the operator implementation path configured in the **FilePath** file. Therefore, the path of the *.json file relative to the path where the **omg** command is executed is fixed to **./kernel_meta**.

- Call the **te::BuildCustomop** function to call the Python function in the operator implementation file to build the operator.

Call the **BuildCustomop** function as follows:

```
te::BuildTeCustomOp(te_bin_info.ddk_version, op.GetName(), FilePath, FuncName, "(i,i,i), s, i, s, f, s",
input_desc.GetShape().GetDim(0),input_desc.GetShape().GetDim(1),input_desc.GetShape().GetDim(2),
input_desc.GetShape().GetDim(3), "float16", axis, operation.c_str(), coeff,KernelName.c_str());
```

Where,

- te_bin_info.ddk_version**: DDK version information (unconfigurable), which will be automatically passed during model conversion
- op.GetName()**: obtaining operator name (unconfigurable),
- FilePath**: relative path of the operator file
- FuncName**: name of the operator implementation function in the operator implementation file
- (i, i, i, i), s, i, s, f, s**: parameter placeholders of the implementation functions in the operator implementation file, where, **i** indicates the integer type, **s** indicates the string type, **f** indicates the single-precision floating-point type, and **o** indicates the PyObject* type. The placeholders must be consistent with the sequence and types of the succeeding parameters, and must be consistent with the definition of the operator implementation function in the operator implementation file.
BuildCustomop calls the operator implementation function based on these parameters and generates the kernel using the TVM mechanism.

8.1.6 Registering an Operator

As the framework manager, Framework provides the **REGISTER_CUSTOM_OP** macro to register an operator based on the specified operator name.

The code of custom operator registration is as follows:

```
REGISTER_CUSTOM_OP("test_layer")
    .FrameworkType(CAFFE)
    .OriginOpType("Test")
```

```
.ParseParamsFn(ParseParamsxx)
.InferShapeAndTypeFn(InferShapeAndTypexx)
.TEBinBuildFn(BuildTeBinxx)
.ImplyType(ImplyType::TVM)
.Formats({DOMI_TENSOR_NC1HWC0}, {DOMI_TENSOR_NC1HWC0})
.WeightFormats({DOMI_TENSOR_FRACTAL_Z, DOMI_TENSOR_NC1HWC0});
```

In the preceding code example:

- **REGISTER_CUSTOM_OP**: Registers a custom operator. Replace **test_layer** with the operator name in the offline model file. The operator name can be random but must not conflict with existing operator names.
- **FrameworkType**: The operator parameter parsing logic varies depending on the framework. Therefore, models under different frameworks require different plug-ins. The plug-in registration code must specify the model framework. Set this parameter to **CAFFE**.
- **OriginOpType**: operator type, which must be the same as the operator type defined in Caffe Prototxt. Otherwise, parsing fails. Find the preset **caffe.proto** file in `/include/inc/custom/proto/caffe/caffe.proto` in the DDK installation path.
- **ParseParamsFn**: Registers the function for model parsing. **ParseParamsxx** has been implemented in [8.1.3 Parsing an Operator](#). This step is required for a plug-in developed for the Caffe framework. If you are rewriting a built-in operator of the Ascend AI processor, skip this step. If the custom operator is not supported by the Ascend AI processor, this step is mandatory.
- **InferShapeAndTypeFn**: Registers the function for shape and class inference. **InferShapeAndTypexx** has been implemented in [8.1.4 Inferring the Output Tensor Description of an Operator](#).
- **TEBinBuildFn**: Registers the TBE operator building function. **BuildTeBinxx** has been implemented in [8.1.5 Building an Operator](#).
- **ImplyType**: Specifies the operator implementation. **ImplyType::TVM** indicates that the operator is a TBE operator.
- **Formats**: Specifies the layout formats of the input data and output data of the operator. The first list is the input data format list, and the second list is the output data format list. If there are multiple inputs, list the layout format of each input data in the first list. For example, if there are two pieces of input data in the NC1HWC0 format, call the **Formats** function as follows:

```
.Formats({DOMI_TENSOR_NC1HWC0, DOMI_TENSOR_NC1HWC0},
{DOMI_TENSOR_NC1HWC0})
```

For details, see the **Formats** in *Framework API Reference*.

- **WeightFormats**: Sets the layout format of operator weight data. For details about the supported data formats, see **WeightFormats** in *Framework API Reference*. For example, the data layout format of the filter of convolution is **fractal_Z**, and the data layout format of the filter of bias is **NC1HWC0**.

If quantization during model conversion is enabled, the constant formats for Framework processing need to be added to this API. Currently, Framework supports the following quantization operators: Conv, FC, and Depthwise Conv. If quantization is enabled for these operators during model conversion, you need to add six **DOMI_TENSOR_NC1HWC0** data formats to the end of the parameter list of the **WeightFormats** API. (During quantization, Framework adds six constants whose data layout format is NC1HWC0. The following is a code sample of the **WeightFormats** API for the convolution operator with quantization enabled:

```
.WeightFormats({DOMI_TENSOR_FRACTAL_Z, DOMI_TENSOR_NC1HWC0, DOMI_TENSOR_NC1HWC0,
DOMI_TENSOR_NC1HWC0, DOMI_TENSOR_NC1HWC0,DOMI_TENSOR_NC1HWC0,
DOMI_TENSOR_NC1HWC0, DOMI_TENSOR_NC1HWC0})
```

8.2 Sample Code

A plug-in sample code is provided: **caffe_reduction_layer.cpp**.

To build the sample file, the following modifications are required:

- Change **FilePath** to the absolute path of the current operator file+name of the operator .py file.
FilePath = "project_path/operator/reduction";
- Change **FuncName** to the name of the operator implementation function in the **reduction.py** file.
FuncName = "reduction"
- Change the value of **KernelName** to that configured in the **reduction.py** file.
KernelName = "Reduction";
- Change the path of the binary file *.o and description file *.json of the operator.

```
te_bin_info.bin_file_path = "./operator/kernel_meta/" + KernelName +
".o";
```

```
te_bin_info.json_file_path = "./operator/kernel_meta/" + KernelName +
".json";A
```

The path is relative to the current project directory. For example, the **Reduction.o** file is in the **operator/kernel_meta** directory under the current project directory.

8.3 Building a Plug-In

With the developed operator plug-in code, build the operator plug-in to generate a binary file and feed the file for OMG to load and register the operator.

Step 1 Log in to the DDK server as the DDK installation user.

Step 2 Modify the **Makefile** file in the **customop_te/plugin** directory of the custom TBE project.

The key information in the **Makefile** file is as follows:

NOTE

If you need to modify the **Makefile** file, avoid introducing spaces to the file.

1. Specify the name of the generated binary file of the operator plug-in.

Example:

```
ll : libcaffe_reduction_layer.so lib_caffe_parser.so
...
libcaffe_reduction_layer.so: $(OBS_customop)
$(CC) -c $(CC_FLAGS) -o proto/caffe/caffe.pb.o proto/caffe/caffe.pb.cc
$(CC) $^ $(LNK_FLAGS) -o $@

lib_caffe_parser.so: $(OBS_no_customop)
$(CC) -c $(CC_FLAGS) -o proto/caffe/caffe.pb.o proto/caffe/caffe.pb.cc
```

```
@if [ -f $(LOCAL_DIR)/proto/caffe/caffe.proto ]; then $(CC) $^ proto/caffe/caffe.pb.o $
(LNK_FLAGS) -o $@; fi;
```

libcaffe_reduction_layer.so is the name of the generated operator plug-in. You can change the name as required.

lib_caffe_parser.so is the library file generated during the parsing of the **caffe.proto** file, and its name must not be changed. For Caffe operators, ensure that all unsupported custom ones in the same model have been defined in [11.1 Defining an Operator in File caffe.proto](#).

2. Specify **TOPDIR** as the DDK installation path.

If the **DDK_PATH** environment variable is not set, set **TOPDIR** to the installation path of the DDK.

If the **DDK_PATH** environment variable is set, the value of **TOPDIR** is the value of **DDK_PATH**.

Example:

```
ifeq ($(DDK_PATH),)
TOPDIR := $HOME/tools/ddk
else
TOPDIR := $(DDK_PATH)
endif
```

3. Specify the path of the header files to be included.

Example:

```
INC_DIR = \
-$(SRC_DIR)\
-$(INCLUDE_DIR)/inc\
-$(INCLUDE_DIR)/inc/custom\
-$(INCLUDE_DIR)/inc/graph\
-$(INCLUDE_DIR)/third_party/protobuf/include\
-$(INCLUDE_DIR)/third_party/json/include\
-$(INCLUDE_DIR)/libc_sec/include \
-/usr/include/python2.7
```

4. Specify the builder path.

Example:

```
CC := LD_LIBRARY_PATH=$(TOPDIR)/uihost/lib:$LD_LIBRARY_PATH g++
```

5. Specify the building parameters.

Example:

```
CC_FLAGS := $(INC_DIR) -g -std=c++11 -fPIC
LNK_FLAGS := \
-L/usr/lib/python2.7/config-x86_64-linux-gnu -lpython2.7\
-L$(TOPDIR)/uihost/lib -lomp -lte_fusion \
-shared

DEMO_LNK_FLAGS := \
-L/usr/lib/python2.7/config-x86_64-linux-gnu -lpython2.7\
-shared
```

Step 3 Build the operator plug-in.

Run the following command in the plug-in directory to build the operator plug-in:

make

The operator plug-in file **libcaffe_reduction_layer.so** is generated in the current directory.

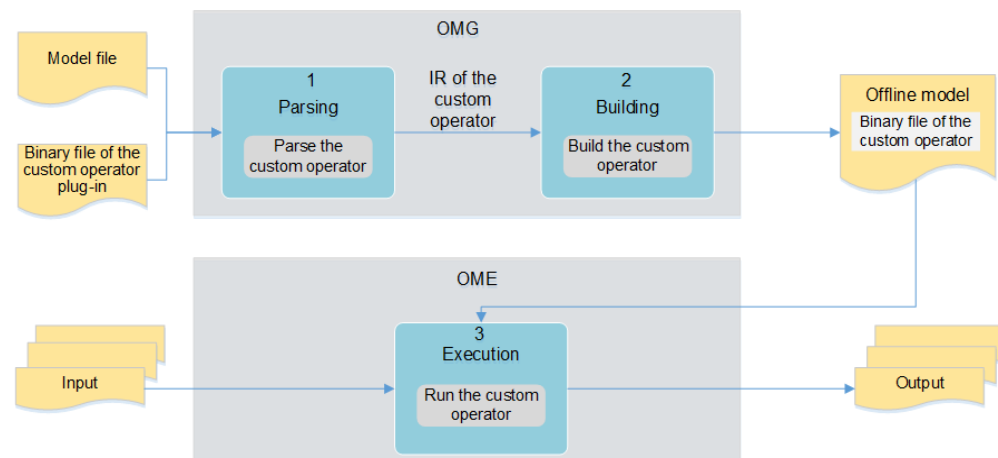
----End

9 Loading a Plug-In for Model Conversion

Principle Description

Figure 9-1 shows the model conversion process when the custom operator plug-in is loaded.

Figure 9-1 Loading a plug-in for model conversion

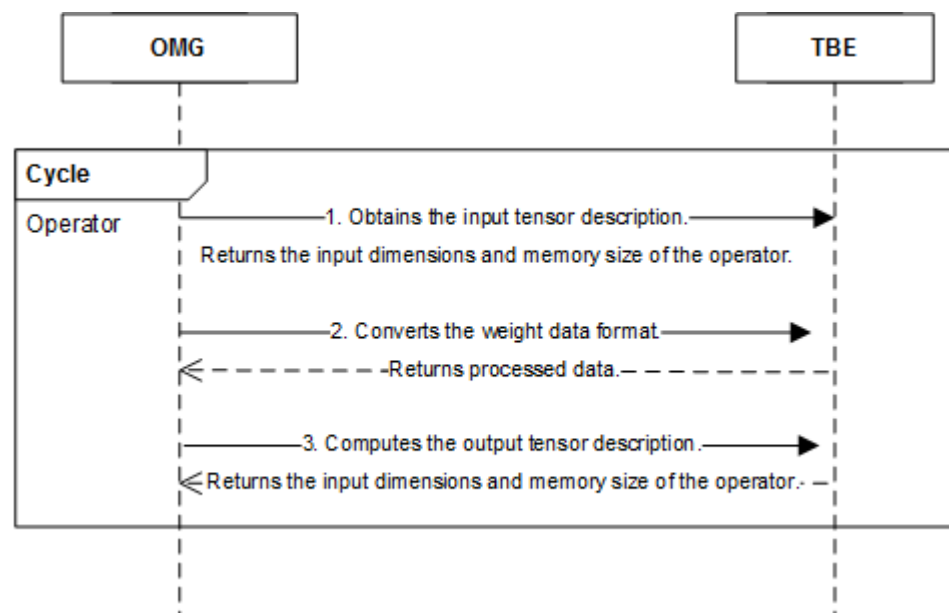


When converting the model file, OMG loads the binary file of the custom operator plug-in to register the operator, and performs operations such as parameter parsing, shape, and type inference. Then, OMG converts the custom operator into a unified IR, and builds the custom operator according to the parsed information. The built kernel is a dedicated kernel placed in the offline model file, which is to be called during model inference.

Specifically, the offline structures specific to operator is generated. Operator generation includes three stages, namely, input tensor description, weight data format conversion, and output tensor description. In the input tensor description, information such as the input dimensions and memory size of an operator is calculated, and the form of input data of the operator is defined in OMG. In weight data format conversion, the weight parameters used by operators are processed, including data format conversion (for example, FP32 to FP16), shape conversion (for example, fractal rearrangement), and data compression. In the output tensor description, information such as the output dimensions and memory size of an operator is calculated.

Figure 9-2 shows the operator generation workflow. During operator generation, the operator acceleration library API of the TBE is used to analyze, determine, and describe the shape of output data. The TBE operator acceleration library API can also be used to convert data formats. OMG receives the IR graph generated by the neural network, describes each node in the IR graph, and parses the inputs and outputs of each operator one by one. OMG analyzes the input source of the current operator, obtains the type of the upper-layer operator, searches the operator library for the output data description of the source operator through the API of the TBE operator acceleration library, and returns the output data information of the source operator to OMG, as the input tensor description of the current operator. Therefore, the description of the input data of the current operator can be learned by analysing the output information of the source operator.

Figure 9-2 Operator generation workflow



Operating Procedure

Step 1 Go to the root directory of the custom operator development project as the DDK installation user.

```
cd $HOME/tools/projects/customop_te/
```

Step 2 Run the following command to convert the model:

```
omg --model=model/deploy_mylenet-1.prototxt --weight=model/
mylenet-1.caffemodel --framework=0 --plugin_path=plugin --output=mylenet
--ddk_version=1.3.T18.B850
```

- **--model**: relative path of the original model file of the MyLeNet network
- **--weight**: relative path of the pre-trained model file of the MyLeNet network
- **--framework**: original framework type
 - 0: Caffe
 - 3: TensorFlow

- **--plugin_path**: directory of the custom operator plug-in
Separate multiple directories with semicolons (;). Semicolons (;) are not allowed in the directory; otherwise, the parsed directory is not as expected.
- **--output**: name of the output model file (configurable)
- **--ddk_version**: version of the matched DDK for running the custom operator.
Check the DDK version in the **\$HOME/tools/che/ddk/ddk/ddk_info** file.

 NOTE

For details about the parameters, see Model Conversion in *Mind Studio User Manual*.

----End

10 Developing an Application

With a converted model, you are able to develop your own applications in no time. For details, see *Application Development Guide (CLI)*.

11 Common Operations

11.1 Defining an Operator in File `caffe.proto`

11.2 Installing the CMake Software

11.1 Defining an Operator in File `caffe.proto`

For a Caffe model, define the operator in the **`caffe.proto`** file as follows. For a TensorFlow model, skip this section.

After an operator is developed, add the definition of the custom operator to the **`caffe.proto`** file preset in the DDK. If the **`caffe.proto`** file has this operator definition already, skip this section.

If there are multiple unsupported custom operators in a model, add related operator definitions at a time by referring to this section. During the implementation of operator plug-ins, related operator parameters are read from the **`caffe.proto`** file based on the operator name, and then the data structures of operators are converted into those supported by the offline model.

Find the preset **`caffe.proto`** file in `/include/inc/custom/proto/caffe/caffe.proto` in the DDK installation directory. You can modify the file and add the definition of the custom operator.

The following describes how to add the definition of the reduction operator (the definition of the reduction operator has been added to the preset **`caffe.proto`** file of the DDK):

Step 1 Add the definition of the reduction operator to **`LayerParameter`**.

Add the definition of the reduction operator to **`LayerParameter`** and set its ID.

```
message LayerParameter {
...
  optional ReductionParameter reduction_param = 136; // The ID must be unique.
...
}
```

Step 2 Add the parameter definition of the reduction operator to the **`caffe.proto`** file.

```
message ReductionParameter {
  enum ReductionOp {          // Operation types supported by the operator
    SUM = 1;                  // Sum the values of all the axes on which the reduce operation is performed.
```

```

    ASUM = 2;           // Sum the calculated absolute values of all axes on which the reduce
operation is performed.
    SUMSQ = 3;          // Sum the squared values of all axes on which the reduce operation is
performed.
    MEAN = 4;           // Average the values of all axes on which the reduce operation is performed.
}
optional ReductionOp operation = 1 [default = SUM]; // Defines the operation of the operator.
optional int32 axis = 2 [default = 0];              // Defines the axis to be reduced.
optional float coeff = 3 [default = 1.0]; // coefficient for output // Scalar, indicating the scaling multiple of
the result
}

```

----End

11.2 Installing the CMake Software

If the CMake software is not installed in Linux, you need to manually deploy the CMake software.

Step 1 Download the CMake software package from <https://cmake.org/download/>.

Download the CMake software package matching the current OS. The version must be 2.8 or later.

For example, if the host OS is **x86_64.centOS**, download the **cmake-3.14.6-Linux-x86_64.tar.gz** software package.

Run the following command to check the OS version:

```
uname -a
```

Step 2 Upload the downloaded CMake software package to any directory on the host as the DDK installation user and decompress the package.

```
tar -zxvf cmake-3.14.6-Linux-x86_64.tar.gz
```

The **cmake-3.14.6-Linux-x86_64** folder is generated in the current directory.

Step 3 Set the CMake environment variable.

```
export PATH=$PATH:/home/xxx/xxx/cmake-3.14.6-Linux-x86_64/bin
```

Replace the preceding path with the actual deployment path of CMake.

Run the **echo \$PATH** command to check whether the environment variable is set successfully.

Step 4 Run the following command to check whether the installation is successful:

```
cmake --version
```

Check the command output.

```
cmake version 3.14.6
CMake suite maintained and supported by Kitware (kitware.com/cmake).
```

The preceding command output indicates that CMake 3.14.6 is successfully deployed.

----End

12 Appendix

[12.1 Change History](#)

12.1 Change History

Release Date	Description
2020-05-30	This issue is the first official release.